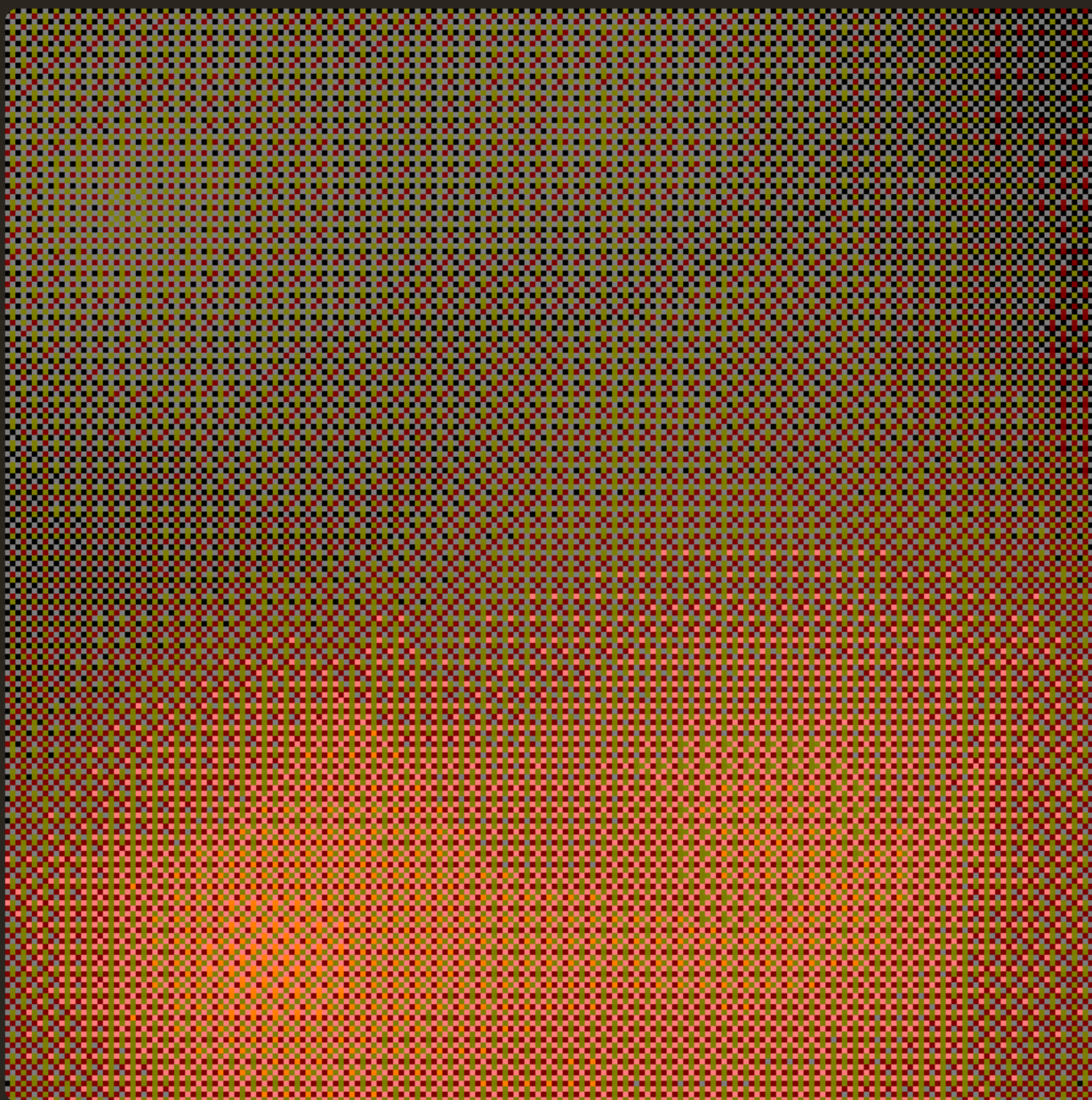


ZKSECURITY

Audit of INTMAX's zERC20

DECEMBER 8TH, 2025 • TECHNICAL REPORT



Introduction

On December 8th, 2025, zkSecurity began a security audit of INTMAX's zERC20 privacy-preserving token system. The audit lasted two weeks with two consultants. We reviewed the [zERC20](#) repository at commit [69537e95](#), corresponding to the [zk-audit-target](#) release.

This is zkSecurity's first audit of the zERC20 system. The zERC20 protocol enables private ERC-20 transfers using a proof-of-burn mechanism: users burn tokens to special addresses derived from Poseidon hashes, then generate zero-knowledge proofs (Nova IVC or Groth16) to withdraw tokens to any recipient without revealing the sender-recipient link.

Scope

The audit focused on circuit correctness and the Solidity application logic integrating the proof-of-burn system.

Circuit Side (R1CS):

We reviewed the following Rust circuits and their associated gadgets:

- `circuits/withdraw.rs`, `circuits/root_transition.rs`, `circuits/burn_address.rs` : Core proof-of-burn step circuits
- `utils/tree/gadgets/` : Merkle proof verification, leaf hash computation, SHA-256 hash chain gadgets
- `utils/poseidon/` : Poseidon hash gadgets and circom-compatible bindings
- `nova/withdraw_nova.rs`, `nova/root_nova.rs` : Nova FCircuit wrappers defining public IO layout
- `groth16/withdraw.rs` : Single-withdrawal Groth16 circuit

We also reviewed witness generation utilities (`utils/tree/merkle_tree.rs`, `utils/tree/incremental_merkle_tree.rs`) and field encoding assumptions (`utils/conversion.rs`) as part of circuit correctness verification.

Solidity Side:

We reviewed the following contracts and libraries:

- `Verifier.sol` : Main proof verification and teleport logic
- `Hub.sol` : Cross-chain Poseidon aggregation hub
- `zERC20.sol` : ERC-20 token with SHA-256 hash chain commitment
- `utils/` : Hash chain, GeneralRecipient, and Poseidon aggregation libraries

The following components were considered out of scope:

- Sonobe prover internals (`nova/params.rs`)
- Generated `Groth16` verifiers
(`verifiers/WithdrawGlobalGroth16Verifier.sol` ,
`verifiers/WithdrawLocalGroth16Verifier.sol`)
- Generated Nova/CycleFold decider contracts
- `liquidity/LiquidityManager.sol` , `liquidity/Adaptor.sol` : Wrap/unwrap flows and OFT compose handling
- `libraries/FeeLib.sol` : Fee calculation math

Summary and Recommendations

The codebase was found to use sound architecture and good coding practices. Security-critical constraints such as overflow prevention in the BN254 field are well implemented. No critical issues were found in the zero-knowledge circuits in scope for this audit. The issues identified relate primarily to documentation, configuration, and privacy model clarity. It is particularly important to document and highlight the expected privacy guarantees so that end users avoid risks related to misuse of the system, such as potential indexer query leakage or anonymity set reduction (see findings below). Minor improvements to documentation and code comments can improve readability and ensure the documentation reflects the current implementation.

Additionally, the security of the smart contract owner keys is critical and should be treated as such. At present, there is no decentralized governance in place, and the owner has essentially unrestricted privileges, including upgrading contracts, minting arbitrary amounts of zERC20s, and burning tokens from any address. As a result, a compromise of the owner key would be catastrophic for the system. The owner key should therefore be protected with the strongest possible operational

security (including clear documentation of the corresponding internal procedures), and the introduction of a more decentralized governance mechanism should be strongly considered to reduce this single point of failure.

Deployment Consideration: LayerZero Security Stack Configuration

While outside the core audit scope, we note that the deployment scripts (`contracts/script`) do not include configuration for LayerZero's [DVN and block confirmation settings](#). zERC20 will deploy to chains with varying finality: BSC's FastFinality settles in seconds, while Arbitrum and Base require a [7-day challenge period](#). Without explicit configuration, LayerZero applies [default settings](#) insufficient for optimistic rollups. If a burn is treated as final before the challenge period ends, a reorg or fraud proof could revert it while the destination mint remains finalized, causing double-spending and inflation.

We recommend configuring finality periods per chain: 180 seconds for BSC (to handle FastFinality failures) and the full 7-day period (302,400 blocks) for Base and Arbitrum. The chosen [LayerZero Security Stack](#) configuration should be documented in detail.

Protocol Overview

The zERC20 system is a privacy-preserving wrapped token that enables unlinkable transfers across EVM chains. The core mechanism is proof-of-burn: senders burn tokens to cryptographically derived “stealth” addresses, and recipients later prove knowledge of the corresponding secret to mint tokens on the same or a different chain. On-chain observers see burns to opaque addresses and mints to recipients, but cannot correlate them. Regular (non-private) transfers are also possible.

The system consists of several coordinated components. The `zERC20` token contract maintains a SHA-256 hash chain that commits to every transfer, providing a compact on-chain summary that zero-knowledge proofs can reference. An off-chain indexer tracks all transfer events and maintains a Poseidon Merkle tree of transfer leaves. When users want to withdraw, they generate either a Nova IVC proof (for batched withdrawals) or a Groth16 proof (for single withdrawals) demonstrating they know secrets corresponding to burns in the tree.

Cross-chain functionality is handled through LayerZero messaging. Each chain runs a `Verifier` contract that validates proofs and tracks withdrawals. A central `Hub` contract aggregates transfer roots from all chains into a global Poseidon tree, enabling recipients to withdraw on any chain using a “global” proof that references the aggregated state. Users can also use “local” proofs that only reference a single chain’s transfer root, trading cross-chain flexibility for faster finality.

The `LiquidityManager` contract handles the wrapping and unwrapping of underlying ERC-20 tokens into zERC20, implementing fee curves that incentivize liquidity rebalancing. An `Adaptor` contract integrates with LayerZero’s OFT standard, enabling composed cross-chain operations where tokens are transferred, unwrapped, and bridged in a single transaction flow.

Proof-of-Burn Mechanism

The proof-of-burn mechanism, based on the concept proposed in [EIP-7503: Zero-Knowledge Wormholes](#) (Private Proof-of-Burn), enables private transfers by breaking the on-chain link between senders and recipients. Instead of transferring directly to a recipient, senders transfer tokens to a cryptographically derived “burn address” that only the intended recipient can later claim. The burn address is an Ethereum address (160 bits) that no one controls directly; tokens sent there are effectively locked until claimed via a zero-knowledge proof.

Burn Address Generation

Burn addresses are derived using the Poseidon hash function with a domain separator:

```
burn_address = truncate_160(poseidon3("burn", recipient_hash, secret))
```

Where:

- `"burn"` is a 4-byte domain separator padded to 32 bytes and interpreted as a field element
- `recipient_hash` is the `GeneralRecipient` hash identifying the intended recipient and target chain
- `secret` is a random field element known only to the sender and shared with the recipient

The result is truncated to 160 bits to produce a valid Ethereum address. Additionally, a 16-bit proof-of-work constraint requires that bits 160-175 of the full Poseidon output are all zero. This is enforced by the circuit (`circuits/burn_address.rs`) and requires the sender to grind through approximately 2^{16} nonce values to find a valid secret. The PoW serves to rate-limit burn address generation and raises the cost of birthday collision attacks on the 160-bit address space from approximately 2^{80} to 2^{96} operations.

Privacy Model

The privacy guarantee is sender-recipient unlinkability: an observer monitoring the blockchain sees transfers to opaque burn addresses and later sees withdrawals to recipient addresses, but cannot determine which burns correspond to which withdrawals.

What is public:

- Burn transactions: sender address, burn address (opaque), and transfer value
- Withdrawal transactions: recipient address, withdrawal value, merkle root used
- The SHA-256 hash chain committing all transfers
- Timing of burns and withdrawals

What is hidden:

- The link between a specific burn and its corresponding withdrawal
- The secret preimage used to derive the burn address
- Which specific leaves in the merkle tree are being claimed (partially hidden; see below)

Trust assumptions and constraints:

- **Contract operators:** The core contracts (`zERC20` , `Verifier` , `Hub` , `LiquidityManager`) are upgradeable. Users must trust the deployer who sets immutable parameters and initial deciders/verifiers, as well as the upgrade/owner roles on each contract. Malicious upgrades could compromise funds or privacy.
- **Amount correlation:** The protocol does not hide transfer amounts. Unique values are trivially linkable between burns and withdrawals. Users must mix standard denominations or use partial withdrawals to achieve meaningful privacy.

- **Indexer privacy:** The indexer service that tracks burns and builds the Merkle tree knows which leaves a user requests proofs for. If the indexer logs query patterns, it can link senders to recipients (see finding on indexer linkability).
- **Trusted setup:** The Groth16 proof path requires a trusted setup ceremony. The current parameter generator uses a fixed seed, which is unsafe for production. A properly conducted multi-party computation ceremony is assumed for production deployment.

Hash Chain

Each zERC20 token maintains a `hashChain` value, which contains a single `uint256`. The hash chain is computed by the below formula:

```
hashChain := SHA256(hashChain || fromAddress || toAddress || value) & ((1<<248) - 1)
```

Note that the hash is truncated to 248 bits from 256 bits for BN254 compatibility.

Additionally, the value will be updated for each transaction in the contract. This will be used when the Verifier contract verifies the soundness of the root transition, where they will be attached with the two ends of the hash chain to ensure that the merkle tree is correctly updated.

Merkle Tree

Each of the verifier contracts maintains a Merkle tree which contains all the transactions for their corresponding zERC20 token, where each node contains `(fromAddress, toAddress, value)` and the tree are hashed using Poseidon.

The Merkle root can be maintained by calling the `proveTransferRoot` function inside the verifier contract. If a prover wants to update the Merkle root, they are required to provide a zero-knowledge proof that the two ends of the hash chain is correct, and it is up-to-date with the zERC20 token.

Zero-Knowledge Proof System

There are two ways to withdraw zERC20 tokens using the Verifier contract, `teleport` and `singleTeleport`. They are respectively using different proof systems: Nova IVC and Groth16.

- `teleport` (using Nova IVC) allows user to withdraw one transaction from the Merkle tree, which has the best privacy among the available transaction

methods to the zERC20 token. One would be able to withdraw an arbitrary amount that is sent to the recipient by proving inclusion of an arbitrary amount of transactions that direct to the recipient on the Merkle tree. Additionally, it is possible to reduce by a certain amount by adding dummy nodes with a negative amount to improve anonymity.

- `singleTeleport` (using Groth16) and a lightweight proof will be used during the process. This would be less private because only one transaction is allowed, but it is much faster to generate a Groth16 proof.

Smart Contracts and Cross-Chain Architecture

LayerZero Primer

LayerZero is a cross-chain messaging protocol that enables contracts on different chains to interact with one another. Each LayerZero-supported chain hosts a LayerZero *endpoint*. An application that wants to use LayerZero for cross-chain messaging can register with that endpoint, specify which remote endpoints it trusts, and use the endpoint to send and receive payloads.

To make integration with the protocol easier, LayerZero provides a pre-built contract that applications can inherit from, and that implements the core interface for calling LayerZero's endpoint: the *OApp* (which stands for "omnichain application").

For tokens, LayerZero offers a special kind of OApp: the *OFT* (omnichain fungible token). An OFT is an ERC20 contract built on top of LayerZero's OApp functionality. On send, it burns locally. On receive, it mints on the destination. During such a transfer, the total supply of an OFT is always preserved across chains. OFTs use the same endpoint path and configuration as any other OApp.

It's important to note that all zERC20 tokens are, in particular, OFT tokens in the sense of LayerZero. This means that zERC20s are not only capable of *private* cross-chain teleports via their proof-of-burn capabilities, but can also serve *public* cross-chain transfers via their built-in LayerZero OFT functionality.

For a more in-depth introduction to the LayerZero protocol, we recommend the following [article](#).

Core Contract Architecture

The smart-contract layer of the zERC20 protocol consists of three primary contracts:

CONTRACT TYPE	PURPOSE	FUNCTIONALITY
zERC20	Actual token implementation	ERC20 compliant, LayerZero OFT, mintable/burnable
Verifier	Verification of ZKPs	Nova verifiers for batch teleports, Groth16 verifiers for single teleports, LayerZero OApp, Tracks proved transfer roots on a given chain and sends these to the Hub
Hub	Cross-chain teleporting	Cross-chain coordination, global aggregation of local transfer roots, LayerZero OApp

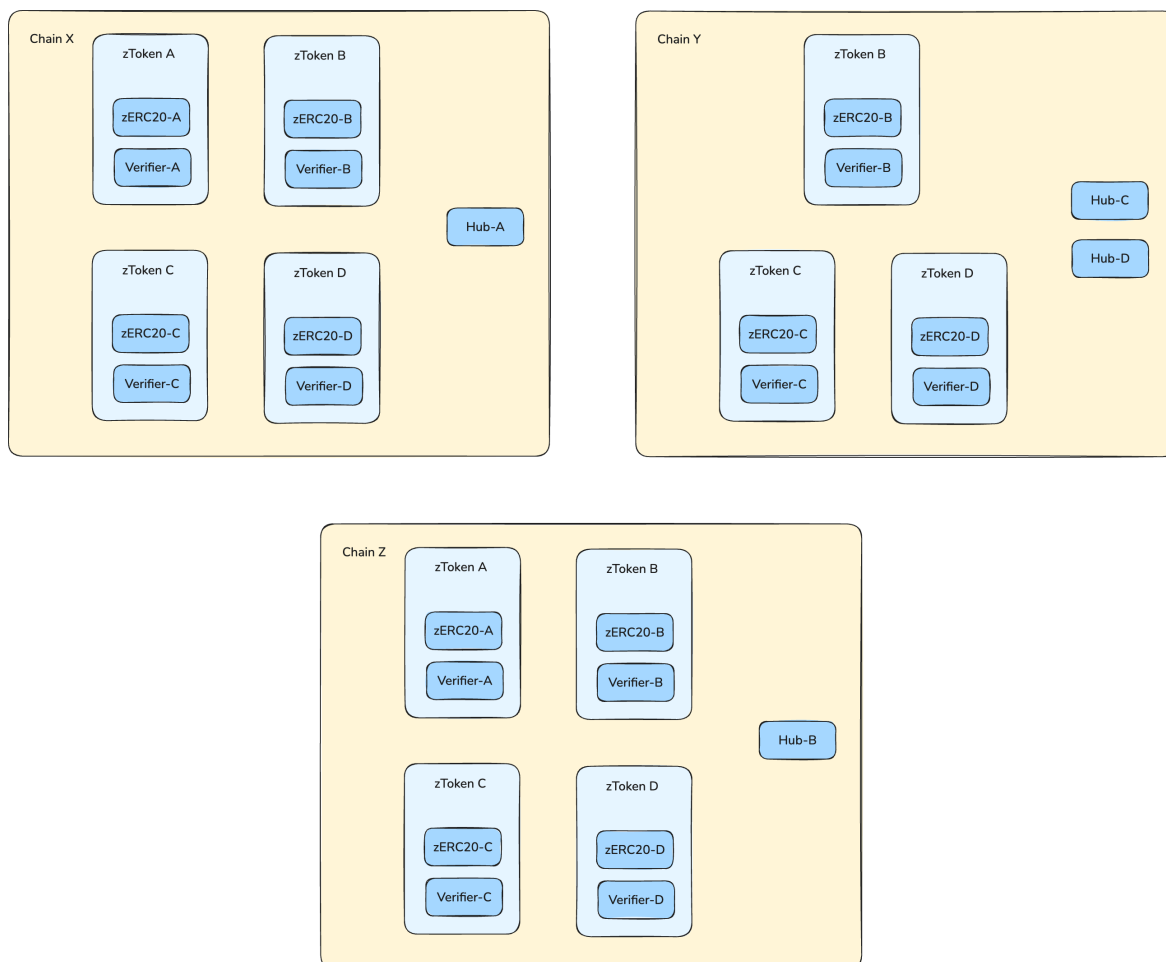
Each zERC20 token will have the following setup:

- One **zERC20** contract per supported chain.
- One **Verifier** contract per supported chain.
- One **Hub** contract, deployed to only a single one of the supported chains.

For example, suppose that there is a zERC20 token called “zUSD”. Further, let’s say that zUSD is supported on three chains: Base, Arbitrum, and BNB Chain. In that case, the zUSD protocol would be comprised of:

- Three **zERC20** contracts.
- Three **Verifier** contracts.
- One **Hub** (e.g., on BNB Chain).

The following diagram illustrates an example setup of the core contract instances that would power an ecosystem of four different zERC20 tokens. Notice that in this example, zToken A is only supported on Chain X and Chain Z, while the other zTokens are supported on all three chains.



A Note on Contract Ownership and Upgradability

`zERC20`, `Verifier`, and `Hub` each set an owner during initialization. That owner is the only entity with control over certain admin functions (see the sections that follow for more info on these functions).

All three follow the UUPS pattern and gate their `_authorizeUpgrade` function with an owner check. In other words, upgrades are possible only through the owner key. Practically, this means that owner compromise not only allows the attacker to call admin functions, but also to swap contract logic at will.

A particular area of concern in the context of upgradable proxies are storage collisions. To reduce storage collision risk across upgrades, each contract follows the ERC-7201 namespaced storage layout. At the time of the audit, this layout choice is the recommended best practice. In particular, it is superior to earlier approaches such as storage gaps.

zERC20

zERC20 represents the main token contract of the zERC20 protocol, i.e., it represents the actual asset. Beyond the standard ERC20 token functionality, its purpose is to record a deterministic history of every transfer. Each token transfer updates a SHA256 hash chain over (from, to, value) and bumps a monotonically increasing index, so the verifier can associate proofs with a specific state of the token.

More specifically, the contract has the following key state variables:

- hashChain and index track the committed transfer history
- totalTeleported tracks cumulative mints that came from “teleport” proofs
- verifier is the verifier contract that’s associated with this token contract

Besides the standard ERC20 interface, zERC20 comes with a teleport function:

```
function teleport(address to, uint256 value) external {
    if (msg.sender != verifier()) revert OnlyVerifier();
    ZERC20Storage storage $ = _getZERC20Storage();
    _mint(to, value);
    $.totalTeleported += value;
    emit Teleport(to, value);
}
```

This function is essentially just a minting function as it mints a specific value to the to address. Beyond that, it only accumulates the totalTeleported and emits a dedicated Teleport event. The most important property of this function is its access control: only the zERC20’s dedicated verifier contract is allowed to call this function. In other words, the only way for users to “teleport” zERC20 tokens to their account is to go through the corresponding verifier contract with a valid proof-of-burn ZKP for the requested teleport.

On the LayerZero side, zERC20 implements the Omnichain Fungible Token (OFT) standard. In other words, in addition to the privacy-preserving “teleport” functionality, every zERC20 token also comes with the option to (publicly) transfer tokens cross-chain via LayerZero’s OFT protocol.

Another important implementation detail is zERC20’s custom transfer hook, _afterTokenTransfer:

```
function _afterTokenTransfer(address from, address to, uint256 value) internal
override(ERC20Upgradeable) {
    if (value > type(uint248).max) revert ValueTooLarge();
```

```

ZERC20Storage storage $ = _getZERC20Storage();
super._afterTokenTransfer(from, to, value);
$.hashChain = ShaHashChainLib.compute($.hashChain, from, to, value);
emit IndexedTransfer($.index++, from, to, value);
}

```

It reverts if a value exceeds the 248-bit bound of the ZKPs, updates the hash chain, and emits `IndexedTransfer` while incrementing `index`. Note that this hook runs for *all* token movements, so that it's guaranteed that the hash chain is always complete.

Besides the verifier-gated `teleport`, the `zERC20` contract comes with several additional access-controlled functions. These are:

- `setVerifier`: Allows the `owner` to set a new `verifier` that is allowed to relay teleport mints.
- `setMinter`: Allows the `owner` to set a “Minter” contract, which is allowed to mint and burn tokens outside of the normal teleport flow.
- `mint`: Allows the `minter` to mint a `value` of tokens to a `to` address.
- `burn`: Allows the `minter` to burn a `value` of tokens from a `from` address.

A few comments are in order: In the future, the `minter` role will most likely be held by a `LiquidityManager` contract, which allows users to wrap/unwrap a given zERC20 against a fixed collateral asset, e.g., USDC. This way, users can “lock” their USDC in the `LiquidityManager` and will receive a corresponding amount of “zUSDC” from the `LiquidityManager`. This zUSDC payout will be facilitated through a call to the `zERC20`'s `mint` function. Similarly, if a user wants to convert their zUSDC back to the original USDC asset, they can “unwrap” their zUSDC in the `LiquidityManager`, which will then call `burn` to burn the zUSDC of the user while paying out the locked, underlying USDC in exchange. However, this user flow is still experimental and was *not* part of the audit scope, which is why we won't explain it here in greater depth.

The only thing worth emphasizing here is the fact that **the `owner` can set themselves as the `minter` and, therefore, mint to and burn from any address at will. For this reason, it's crucial that the `owner` key is carefully protected.**

As already explained in the previous sections, each zERC20 token will be realized as one-per-supported-chain deployments of the `zERC20` contract. So, for example, a “zUSD token” with cross-chain support for Chain X, Y, and Z, would

need one `zERC20` deployment on each of the three chains.

Verifier

Every `zERC20` instance comes with a corresponding `Verifier` contract. The main purpose of this contract is to gate teleports. More specifically, for users to be able to mint `zERC20` tokens via a teleport, they need to provide a valid proof-of-burn ZKP to the `Verifier`.

The `Verifier` offers users two different ways to do teleports:

1. The `teleport` function handles multi-note Nova proofs, and allows users to do batch withdrawals.
2. The `singleTeleport` function handles Groth16 single-note proofs, which allows users to withdraw funds corresponding to a single burn.

After successful verification, both of these functions internally call `zERC20.teleport` to mint `zERC20` tokens to the recipient.

It's important to note that the hinted root can be either a *local* proved root or a Hub-derived *global* root. To indicate whether the provided ZKP references a teleport that is local (i.e., happens on the same chain) or global (i.e., cross-chain), users specify the boolean input `isGlobal` of `teleport` or `singleTeleport`, respectively.

On the LayerZero side, the `Verifier` extends LayerZero's `OAppUpgradeable`. The main reason for this is that every `Verifier` needs to sync their local state with the global state that is tracked on the `Hub`. More specifically, each `Verifier` contract periodically sends its latest proved transfer root to the `Hub` via LayerZero. In the opposite direction, each verifier's `_lzReceive` hook periodically accepts the global root from the `Hub`. As explained earlier, with these global roots synced, users can teleport cross-chain by specifying `isGlobal = true` when calling `teleport` or `singleTeleport`.

Lastly, the `Verifier` contract sets a contract owner upon initialization, who can upgrade the contract and has access to certain admin functionality. Concretely, the owner can call `activateEmergency` to pause the verifier contract's teleport functionality. Similarly, the owner can call `deactivateEmergency` to return to normal operation. Moreover, the owner can reset the Nova/Groth16 ZKP verification contracts that the `Verifier` is using under the hood.

Hub

For a given zERC20 token, the `Hub` is the central LayerZero OApp that keeps one leaf per registered token instance and periodically aggregates all leaves (i.e., the local transfer roots) into a global Merkle root that the local `Verifier` contracts consume to stay in sync with the global state. Besides the local transfer roots that it stores in the `transferRoots` array, it also stores the token info (`chainId` , `eid` , `verifier` address, and `token` address) for each local instance of the zERC20 token in question. The Poseidon-aggregated global root can hold a maximum of 64 leaves, i.e., 64 is the maximum number of chains the `Hub` can support for a given zERC20 token.

Like the `Verifier` and `zERC20` contracts, the `Hub` contract also sets an owner upon deployment. After deployment of the hub, this owner can register the local instances of the given zERC20 token via `registerToken` , which fixes the token instance's leave position in the global Merkle tree. Furthermore, the owner can update the token info of a given instance, using the `updateToken` function.

As previously explained, the hub periodically receives new local transfer roots from the `Verifier` contracts. These incoming messages from the verifiers land in `_lzReceive` , which updates the `transferRoots` array of the `Hub` contract. However, the hub does not only *receive* messages from the verifiers, it also *broadcasts* messages to them. More specifically, the hub periodically broadcasts the aggregated `globalRoot` to each supported chain's `Verifier` instance via LayerZero. In other words, each chain's `Verifier` feeds its latest proved root upward, the `Hub` aggregates them, and the aggregated roots flow back down so verifiers can serve global, cross-chain teleports.

Findings

Below are listed the findings found during the engagement. High severity findings can be seen as so-called "priority 0" issues that need fixing (potentially urgently). Medium severity findings are most often serious findings that have less impact (or are harder to exploit) than high-severity findings. Low severity findings are most often exploitable in contrived scenarios, if at all, but still warrant reflection. Findings marked as informational are general comments that did not fit any of the other criteria.

ID	COMPONENT	NAME	RISK
#01	docs/contract_spec.md	Indexer Service Can Link Senders to Recipients	Low
#02	docs/zkp_spec.md	Collision attack complexity not as high as documented	Low
#03	contracts/src/Verifier.sol	Nova Proof Leaks Last Leaf Index in Calldata	Low
#04	zkp/src/nova/withdraw_nova.rs, zkp/src/bin/generate_circuit_artifacts.rs	Minor Code Consistency Issues	Informational
#05	docs/contract_spec.md, docs/architecture.md	Documentation Does Not Match Implementation	Informational
#06	contracts/src/Verifier.sol, contracts/src/Hub.sol, contracts/src/zERC20.sol	Lack of Two- Step Process for Ownership Transfer	Informational

#01 - Indexer Service Can Link Senders to Recipients

Severity: Low **Location:** docs/contract_spec.md

Description. The zERC20 protocol documentation describes sender-recipient unlinkability as a core privacy guarantee. In docs/architecture.md:7, the system is described as providing “privacy-preserving transfers” where “a sender burns zERC20 to a stealth address and a recipient later mints on the same or another chain without linkable on-chain metadata.” The trust model in docs/contract_spec.md:5-6 explicitly enumerates trusted actors as “(a) the deployer who sets immutable parameters and initial deciders/verifiers, and (b) the upgrade/owner roles on each upgradeable contract.”

However, the protocol relies on an indexer service that is not included in this trust model. As described in docs/payment.md:51, recipients “query the indexer for matching transfers” to discover burns destined for them. The indexer backend observes all on-chain transfer events including the sender address, burn address, and transfer value. When a recipient queries for their burns, the indexer learns which burn addresses correspond to which recipients. Combined with the sender information from the original burn transactions, the indexer can reconstruct the full sender-to-recipient mapping.

The documentation does not clearly specify whether the indexer is intended to run as an external hosted service or locally by the user. The indexer source code is included in the repository (indexer/) and appears configurable via environment variables, suggesting local deployment is possible. However, it is unclear whether the default configuration points to an external backend operated by a third party. Users expecting end-to-end privacy may not realize they need to run their own indexer infrastructure to avoid trusting an external operator with their transaction linkage data.

Impact. The privacy model is weaker than the documented trust assumptions suggest. If users rely on an externally operated indexer, they must trust that operator not to log or leak the sender-recipient linkage data. This represents a single point of privacy failure that is not disclosed in the trust model documentation.

Recommendation. Update the documentation to clearly specify the indexer’s role in the trust model. The documentation should explain whether the default deployment uses an external indexer service or expects users to run their own

instance. If an external service is the default, add the indexer operator to the trusted actors list in `docs/contract_spec.md`. Provide clear instructions for users who wish to run a local indexer to preserve full privacy, and clarify that users who do not run their own indexer are trusting the operator with sender-recipient linkability information.

Client Response. The client acknowledged the finding and addressed it in commit `0d2bc86` by adding documentation to `docs/contract_spec.md` clarifying that the indexer is privacy-sensitive, explaining what information it can observe and link, and recommending that users who want full unlinkability run their own indexer instance.

#02 - Collision attack complexity not as high as documented

Severity: Low **Location:** docs/zkp_spec.md

Description. The effort to generate a collision attack is documented to be 2^{96} [here](#). However, since EOA keys can be generated efficiently while burn addresses require PoW, an attacker can exploit this asymmetry to reduce the attack complexity to $\approx 2^{89}$ as follows.

Instead of searching for a pair of `(recipient, secret)` that end up with the same burn addresses, an adversary can generate two sets, one with $n = 2^{88}$ EOA private keys and one with $m = 2^{72}$ valid burn addresses. In that case, the probability that there exists a pair of colliding addresses from each set would be

$$1 - \left(1 - \frac{m}{2^{160}}\right)^n \approx 1 - \frac{1}{e} = 63\%$$

and the total effort is only $2^{88} + 2^{16} \times 2^{72} = 2^{89}$.

Impact. A successful collision allows an attacker to find a burn address that is also a valid Ethereum address they control. This enables double-spending: the attacker can claim funds via a zero-knowledge proof and transfer them as a normal ERC-20 transaction. While the attack requires approximately 2^{89} operations (which is still computationally prohibitive with current technology) the security margin of 89 bits falls below NIST's recommended minimum of 112 bits for long-term security. The documentation should be corrected to reflect the actual security level.

Recommendation. Update the documentation to accurately state the collision resistance is approximately 2^{89} , not 2^{96} . If a higher security margin is desired, consider increasing the proof-of-work requirement from 16 bits to 24 bits, which would raise the attack complexity to approximately 2^{93} . Alternatively, document the current security level as acceptable given the computational infeasibility of 2^{89} operations with present-day hardware.

Client Response. Client has added a security note regarding the PoW complexity [here](#).

#03 - Nova Proof Leaks Last Leaf Index in Calldata

Severity: Low **Location:** contracts/src/Verifier.sol

Description. The Nova proof exposes `proof_[7]` (final `leaf_index_with_offset`) publicly in `Verifier.teleport`, but the contract doesn't enforce any padding rule. If a prover stops early (low final index), this can leak metadata about how far into the tree they aggregated, potentially shrinking the anonymity set. For instance, one could recognize an early transaction if claimed later when the tree is bigger.

The CLI mitigates this by adding dummy steps to push the final index near the tree limit (`cli/src/proof/batch.rs` lines 75 to 83), but this is not enforced on-chain. Unfortunately, one cannot simply make this parameter private without changing the Nova public state layout and keys, because the final folded state is part of what the on-chain decider verifies.

One could require the final index to equal $(1 \ll \text{DEPTH}) - 1$, which would reduce this risk at the cost of extra proving time. However, enforcement would require maintaining tree depth constants (`TRANSFER_TREE_HEIGHT`, `GLOBAL_TRANSFER_TREE_HEIGHT`) in the contract in sync with the circuit parameters, which is cumbersome.

Impact. Alternative provers that do not implement dummy step padding may inadvertently leak timing metadata, reducing user privacy. Users of the official CLI are protected by convention.

Recommendation. Document this requirement clearly in the protocol documentation. Update the on-chain comment at `Verifier.sol` in line 320 to warn that `proof_[7]` is visible in calldata and that provers must pad to the maximum index to preserve privacy.

Client Response. The client acknowledged the finding and addressed it in commit [68b0862](#) by updating the comment in `Verifier.sol` to warn that `lastLeafIndex` is visible in calldata and provers should pad to the maximum index, and by adding documentation to `docs/contract_spec.md` explaining the Nova proof padding requirement and that alternative provers should implement the same convention as the CLI.

#04 - Minor Code Consistency Issues

Severity: Informational **Location:** `zkp/src/nova/withdraw_nova.rs`,
`zkp/src/bin/generate_circuit_artifacts.rs`

Description. The codebase contains minor inconsistencies that do not affect correctness but reduce readability.

First, the `is_dummy` field uses different types across Nova circuits. In `withdraw_nova.rs:30`, it is declared as a field element `F`, requiring explicit zero/one checking during witness allocation. In contrast, `root_nova.rs:27` declares it as a native `bool`, which is simpler and more idiomatic. The semantics are equivalent, but the inconsistency may confuse developers reviewing or extending the circuits.

Second, log messages in `generate_circuit_artifacts.rs` contain incorrect labels, printing “Groth16” when generating Nova artifacts or vice versa.

Impact. These issues do not affect security or correctness but reduce code clarity and may cause confusion during development or debugging.

Recommendation. Harmonize the `is_dummy` type to use `bool` consistently across all Nova circuits. Correct the log messages in `generate_circuit_artifacts.rs` to accurately reflect the artifact type being generated.

Client Response. The client acknowledged the finding and addressed it in commit [9b1c300](#) by changing the `is_dummy` field from a field element to `bool` in `withdraw_nova.rs` and updating all usages accordingly, and by correcting the swapped log messages in `generate_circuit_artifacts.rs`.

#05 - Documentation Does Not Match Implementation

Severity: Informational **Location:** docs/contract_spec.md, docs/architecture.md

Description. The protocol documentation contains inconsistencies with the actual implementation.

First, docs/contract_spec.md describes a Minter contract with depositNative, depositToken, withdrawNative, and withdrawToken functions for bridging native and ERC-20 liquidity into zERC20. However, this contract does not exist in the codebase. The actual implementation uses LiquidityManager.sol with wrap and unwrap functions instead.

Second, the documentation emphasizes the privacy-preserving teleport mechanism without mentioning that zERC20 inherits from OFTCoreUpgradeable, which exposes a standard LayerZero send() function for non-private cross-chain transfers. This alternative path is already used internally (e.g., cli/src/proof/unwrap.rs:248-251) but is not documented. Users seeking simpler cross-chain transfers without privacy requirements may not realize this option exists.

Impact. Developers and integrators relying on the documentation may implement incorrect interfaces or miss available functionality.

Recommendation. Update contract_spec.md to reflect the actual LiquidityManager interface. Document the non-private OFT send() path as an alternative to the privacy-preserving teleport flow, clarifying when each is appropriate.

Client Response. The client addressed both issues. In commit [da20fcc](#), the documentation in docs/contract_spec.md was updated to replace references to the non-existent Minter contract with the actual LiquidityManager interface, including its wrap and unwrap functions. The OFT send() path is now documented in the Adaptor and Liquidity Entry/Exit Flow sections, explaining how users can use it for cross-chain transfers.

#06 - Lack of Two-Step Process for Ownership Transfer

Severity: Informational **Location:** contracts/src/Verifier.sol, contracts/src/Hub.sol, contracts/src/zERC20.sol

Description. Through `OAppCoreUpgradable`, the `Verifier`, `Hub`, and `zERC20` contracts inherit OpenZeppelin's `Ownable` contract, which implements a single-step ownership transfer mechanism.

Impact. If ownership is accidentally transferred to an incorrect address, this would result in a permanent loss of administrative control.

Recommendation. The `Ownable2Step` pattern provides a more secure ownership transfer mechanism by requiring the new owner to accept the transfer in a separate transaction, reducing the risk of accidental transfers. Consider importing `Ownable2Step` instead of `Ownable`. Also, if ownership is never supposed to be renounced, consider overriding `renounceOwnership` to always revert to avoid the risk of accidentally renouncing admin control.

Client Response. Acknowledged. The team decided not to adopt a two-step ownership transfer, as doing so would require manual modifications to a third-party dependency contract.